Intel SGX SDK 구현체 상세 분석

개요

Intel Software Guard Extensions (SGX)는 애플리케이션 개발자가 선택된 코드와 데이터를 공개나 수정으로부터 보호할 수 있도록 하는 Intel 기술입니다. 이 문서는 Linux용 Intel SGX SDK의 구현체를 상세히 분석하여 각 프로세스와 컴포넌트의 역할을 설명합니다.

전체 아키텍처

Intel SGX 소프트웨어 스택은 다음 세 가지 주요 구성 요소로 이루어져 있습니다:

1. **Intel SGX Driver**: 하드웨어와의 인터페이스 제공
2. **Intel SGX SDK**: 개발 도구 및 라이브러리
3. **Intel SGX PSW (Platform Software)**: 런타임 서비스

주요 디렉토리 구조 및 역할

1. SDK (Software Development Kit) - `/sdk`

SDK는 SGX 애플리케이션 개발을 위한 도구와 라이브러리를 제공합니다.

1.1 핵심 컴포넌트

TRTS (Trusted Runtime System) - `/sdk/trts`

- Enclave 내부에서 실행되는 신뢰할 수 있는 런타임 시스템
- Enclave 초기화, ECALL/OCALL 처리, 예외 처리 담당
- 주요 파일:

- `trts.cpp` : 메인 런타임 로직
- `trts_ecall.cpp` : Enclave 호출 처리
- `trts_ocall.cpp` : 외부 호출 처리
- `init_enclave.cpp` : Enclave 초기화

Edger8r - `/sdk/edger8r`

- EDL(Enclave Definition Language) 파서 및 코드 생성기
- Enclave와 애플리케이션 간의 인터페이스 자동 생성
- ECALL/OCALL 함수의 프록시 코드 생성

Sign Tool - `/sdk/sign_tool`

- Enclave 서명 도구
- 개발자의 개인키로 Enclave 바이너리에 서명
- 메타데이터 및 측정값(measurement) 생성

1.2 라이브러리 컴포넌트

Trusted Libraries (Enclave 내부용)

- `tlibc` : Enclave용 C 표준 라이브러리
- `tlibcxx` : Enclave용 C++ 표준 라이브러리
- `tlibcrypto` : Enclave용 암호화 라이브러리
- `pthread` : Enclave용 스레드 라이브러리
- `tseal` : 데이터 봉인/해제 라이브러리

Untrusted Libraries (애플리케이션용)

- `ukey_exchange` : 키 교환 라이브러리
- `utls` : TLS 통신 라이브러리

1.3 보안 기능

Protected File System - `/sdk/protected_fs`

- 암호화된 파일 시스템 제공
- Enclave 내부에서 안전한 파일 I/O 수행

Sealing - `/sdk/tseal`

- 데이터를 플랫폼에 바인딩하여 암호화
- 동일한 Enclave에서만 해독 가능한 데이터 생성

2. PSW (Platform Software) - `/psw`

PSW는 SGX 런타임 서비스를 제공하는 플랫폼 소프트웨어입니다.

2.1 URTS (Untrusted Runtime System) - `/psw/urts`

주요 역할:

- Enclave 생성, 로딩, 관리
- 메모리 관리 및 페이지 테이블 설정
- Launch Control 정책 적용

핵심 파일:

- `enclave.cpp` : Enclave 객체 관리
- `loader.cpp` : Enclave 바이너리 로딩
- `launch_checker.cpp` : Launch 정책 검증

2.2 Architecture Enclaves - `/psw/ae`

Launch Enclave (LE) - `/psw/ae/le`

- Enclave 실행 허가 결정

- Launch Token 생성 및 검증
- Flexible Launch Control 지원

Provisioning Enclave (PvE) - `/psw/ae/pve`

- 플랫폼 프로비저닝 수행
- EPID 키 생성 및 관리

Quoting Enclave (QE) - `/psw/ae/qe`

- Remote Attestation을 위한 Quote 생성
- EPID 서명 생성

Platform Certification Enclave (PCE) - `/psw/ae/pce`

- 플랫폼 인증서 관리
- ECDSA 기반 attestation 지원

2.3 AESM Service - `/psw/ae/aesm_service`

기능:

- Architecture Enclave들의 관리 및 조정
- 네트워크 통신을 통한 Intel 서비스 연동
- Launch Token 및 Quote 요청 처리

3. 공통 컴포넌트 - `/common`

SE Wrapper - `/common/se_wrapper`

- 플랫폼별 시스템 호출 래퍼
- 크로스 플랫폼 호환성 제공

Include Files - `/common/inc`

- 공통 헤더 파일 및 데이터 구조 정의

- SGX 명령어 및 상수 정의

주요 프로세스 분석

1. Enclave 생성 프로세스

```
1. 애플리케이션이 sgx_create_enclave() 호출
2. URTS가 Enclave 바이너리 로딩
3. 메모리 할당 및 페이지 설정
4. Launch Enclave를 통한 허가 검증
5. Enclave 초기화 및 측정값 계산
6. EINIT 명령어로 Enclave 활성화
```

2. ECALL/OCALL 처리 프로세스

ECALL (Enclave Call) 프로세스:

```
1. 애플리케이션에서 ECALL 함수 호출
2. Edger8r 생성 프록시 코드 실행
3. 매개변수 마샬링 및 검증
4. EENTER 명령어로 Enclave 진입
5. TRTS에서 함수 디스패치
6. 실제 Enclave 함수 실행
7. EEXIT으로 결과 반환
```

OCALL (Outside Call) 프로세스:

```
1. Enclave 내부에서 OCALL 함수 호출
2. TRTS가 OCALL 요청 처리
3. EEXIT으로 Enclave 외부로 나감
4. URTS가 OCALL 디스패치
5. 애플리케이션 함수 실행
6. EENTER로 Enclave 재진입
7. 결과를 Enclave로 반환
```

3. Remote Attestation 프로세스

```
1. 애플리케이션이 sgx_create_report() 호출
2. EREPORT 명령어로 Report 생성
3. AESM Service에 Quote 요청
4. Quoting Enclave가 Quote 생성
5. Intel Attestation Service로 검증 요청
6. 검증 결과 수신 및 처리
```

4. 데이터 봉인(Sealing) 프로세스

```
1. 애플리케이션이 sgx_seal_data() 호출
2. 봉인 정책 결정 (MRENCLAVE 또는 MRSIGNER)
3. EGETKEY 명령어로 봉인 키 획득
```

4. AES-GCM으로 데이터 암호화

5. 봉인된 데이터 구조 생성

빌드 시스템 분석

1. 메인 Makefile 구조

주요 타겟:

- `sdk` : SDK 컴포넌트 빌드
- `psw` : PSW 컴포넌트 빌드
- `preparation` : 서브모듈 및 의존성 준비
- `clean` : 빌드 아티팩트 정리

2. 빌드 옵션

USE_OPT_LIBS 옵션:

- `0` : SGXSSL + 오픈소스 문자열/수학 라이브러리
- `1` : IPP 암호화 + 오픈소스 문자열/수학 라이브러리 (기본값)
- `2` : SGXSSL + 최적화된 문자열/수학 라이브러리
- `3` : IPP 암호화 + 최적화된 문자열/수학 라이브러리

보안 완화 옵션:

- `MITIGATION-CVE-2020-0551=LOAD` : 전체 완화 적용
- `MITIGATION-CVE-2020-0551=CF` : 간접 분기 및 반환 완화만 적용

3. 패키지 생성

DEB 패키지 (Ubuntu/Debian):

- `deb_psw_pkg` : PSW 패키지 생성
- `deb_local_repo` : 로컬 저장소 생성

RPM 패키지 (RHEL/CentOS):

- `rpm_psw_pkg` : PSW 패키지 생성
- `rpm_local_repo` : 로컬 저장소 생성

외부 의존성

1. 암호화 라이브러리

- **Intel IPP Crypto**: 최적화된 암호화 연산
- **SGX SSL**: OpenSSL 기반 SGX용 SSL/TLS

2. 개발 도구

- **OCaml**: Edger8r 구현 언어
- **Protobuf**: 데이터 직렬화
- **OpenMP**: 병렬 처리 지원

3. DCAP (Data Center Attestation Primitives)

- ECDSA 기반 attestation 지원
- Quote Verification Library
- PCK Certificate 관리

보안 고려사항

1. 메모리 보호

- Stack Canary 및 FORTIFY_SOURCE 적용
- DEP (Data Execution Prevention) 활성화
- ASLR (Address Space Layout Randomization) 지원

2. 사이드 채널 공격 완화

- CVE-2020-0551 완화 옵션 제공
- 간접 분기 및 반환 주소 보호
- 메모리 접근 패턴 보호

3. 코드 무결성

- Enclave 서명 및 검증
- 측정값 기반 무결성 확인
- Launch Control 정책 적용

개발 워크플로우

1. 개발 환경 설정

```
# 의존성 설치
sudo apt-get install build-essential ocaml ocamlbuild automake autoconf


# 소스 코드 준비
git clone https://github.com/intel/linux-sgx.git
```

```
cd linux-sgx

make preparation
```

2. SDK 빌드

```
# 기본 빌드

make sdk


# 디버그 빌드

make sdk DEBUG=1


# 완화 옵션 적용

make sdk MITIGATION-CVE-2020-0551=LOAD
```

3. 애플리케이션 개발

```
# EDL 파일 작성

# Enclave 코드 구현

# 애플리케이션 코드 구현

# Makefile 설정

make
```

성능 최적화

1. 컴파일러 최적화

- -O2 최적화 레벨 적용
- 함수 및 데이터 섹션 분리
- 링크 타임 최적화 지원

2. 라이브러리 최적화

- Intel IPP를 통한 암호화 연산 가속
- 최적화된 문자열 및 수학 함수
- SIMD 명령어 활용

3. 메모리 관리

- 동적 힙 확장 지원
- 스레드 풀 관리
- 메모리 단편화 최소화

이 문서는 Intel SGX SDK의 복잡한 구현체를 체계적으로 분석하여 각 컴포넌트의 역할과 상호작용을 명확히 설명합니다. 개발자들이 SGX 기술을 이해하고 활용하는 데 도움이 되기를 바랍니다.